



复旦大学人工智能课程AI-Basics Courses

人工智能编程基础

Fundamentals of Artificial Intelligence Programming

计算机科学技术学院

周雅倩
zhouyagqian@fudan.edu.cn



1

References

- <http://speech.ee.ntu.edu.tw/~hylee/ml/2023-spring.php>
 - Partial contents are copied from the slides of 李宏毅教授

2



RNN和语音识别
RNN and Automatic Speech Recognition

RNN和语音识别

RNN and Automatic Speech Recognition

- RNN和LSTM
- 语音识别

3



RNN和语音识别
RNN and Automatic Speech Recognition

例子

- Slot Filling



4

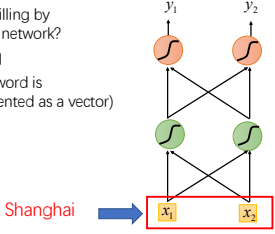


RNN和语音识别
RNN and Automatic Speech Recognition

例子

Solving slot filling by Feedforward network?

Input: a word
(Each word is represented as a vector)



5

1-of-N encoding

How to represent each word as a vector?

1-of-N Encoding lexicon = {apple, bag, cat, dog, elephant}

The vector is lexicon size.

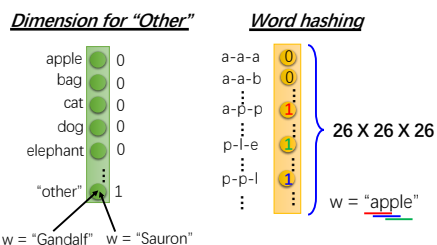
Each dimension corresponds to a word in the lexicon

The dimension for the word is 1, and others are 0

apple	=	[1 0 0 0 0]
bag	=	[0 1 0 0 0]
cat	=	[0 0 1 0 0]
dog	=	[0 0 0 1 0]
elephant	=	[0 0 0 0 1]

6

Beyond 1-of-N encoding



7

Embedding

• **Embedding 层** 是深度学习用于将离散的符号（如单词、字符、类别等）映射到连续向量空间的核心组件。它在自然语言处理（NLP）、推荐系统等领域中广泛应用。

• 将离散符号映射到连续向量：

- 将单词、字符、类别等离散符号映射为固定长度的连续向量。

- 例如，将单词 "cat" 映射为一个 100 维的向量。

• 捕捉语义信息：

- 通过训练，Embedding 层可以学习到符号之间的语义关系。

- 例如，"cat" 和 "dog" 的向量在向量空间中距离较近，而 "cat" 和 "car" 的向量距离较远。

Embedding 层的训练

• **随机初始化**：Embedding 层的权重通常是随机初始化的。

• **训练更新**：在模型训练过程中，Embedding 层的权重会通过反向传播更新。

• **预训练嵌入**：可以使用预训练的嵌入向量（如 Word2Vec、GloVe）初始化 Embedding 层。

8

Example Application

Solving slot filling by Feedforward network?

Input: a word

(Each word is represented as a vector)

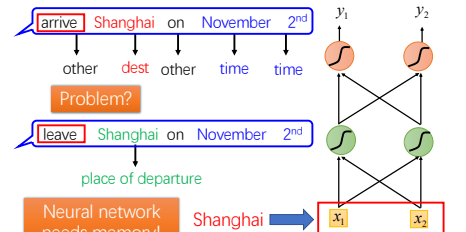
Output:

Probability distribution that the input word belonging to the slots

Shanghai $\rightarrow$ x_1 x_2

9

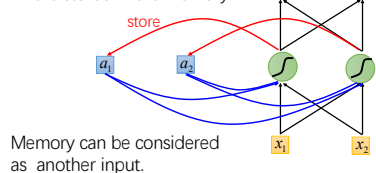
Example Application



10

Recurrent Neural Network (RNN)

The output of hidden layer are stored in the memory.

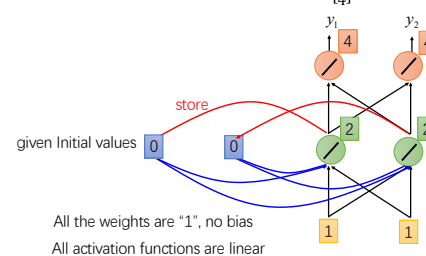


11

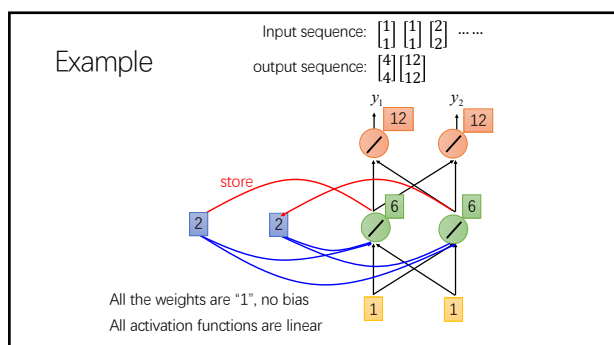
Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$...

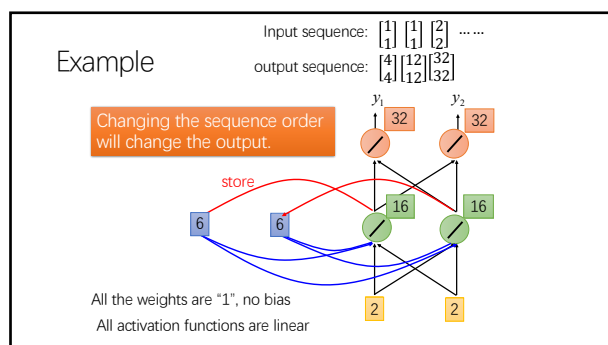
output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$



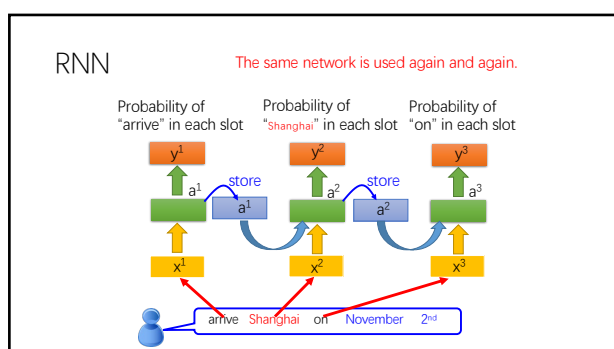
12



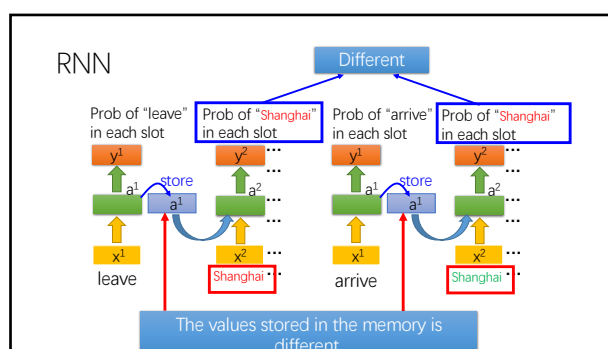
13



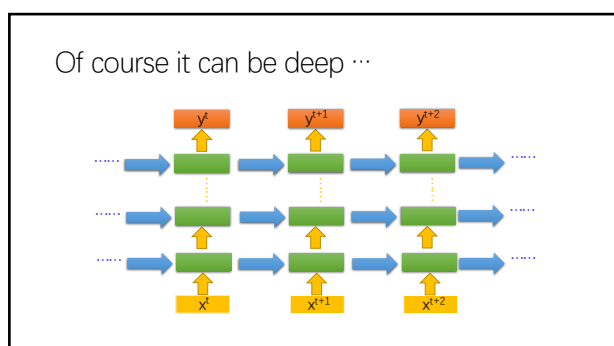
14



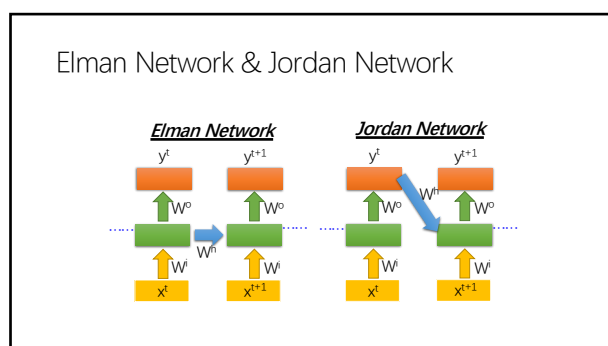
15



16

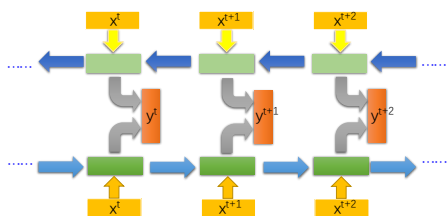


17



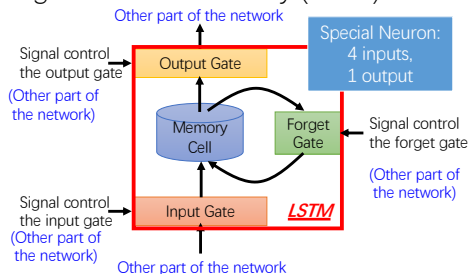
18

Bidirectional RNN

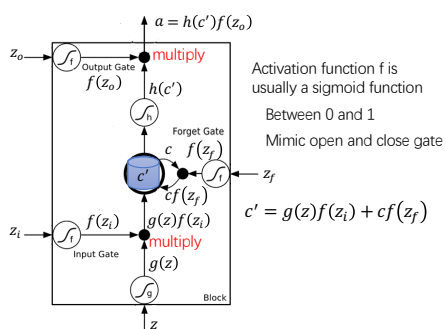


19

Long Short-term Memory (LSTM)



20



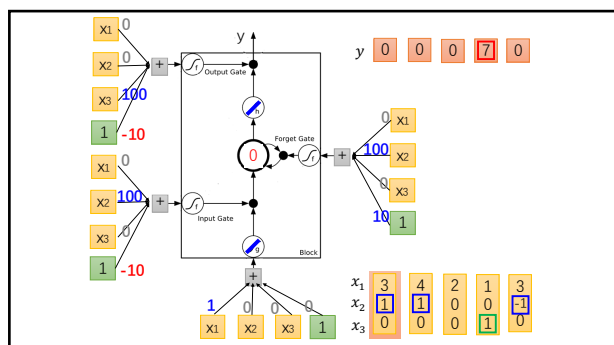
21

LSTM - Example

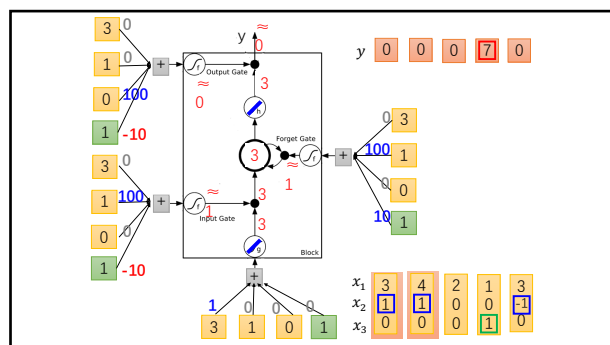
	0	0	3	3	7	7	7	0	6
x_1	1	3	2	4	2	1	3	6	1
x_2	0	1	0	1	0	0	-1	1	0
x_3	0	0	0	0	0	1	0	0	1
y	0	0	0	0	0	7	0	0	6

When $x_2 = 1$, add the numbers of x_1 into the memory
 When $x_2 = -1$, reset the memory
 When $x_3 = 1$, output the number in the memory.

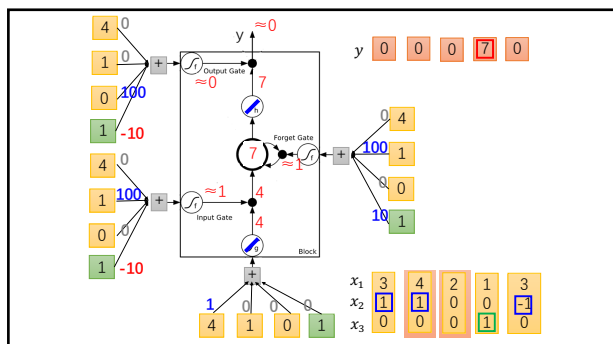
22



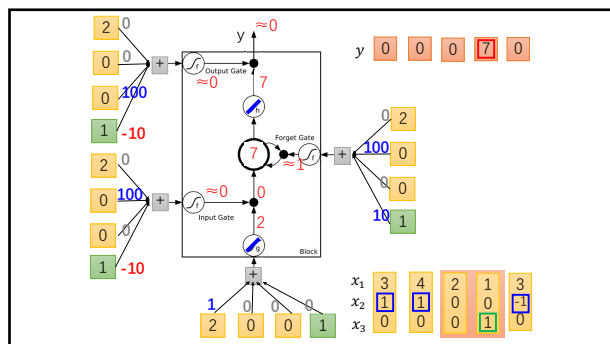
23



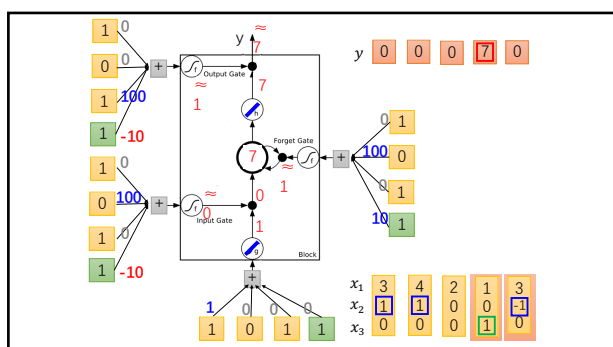
24



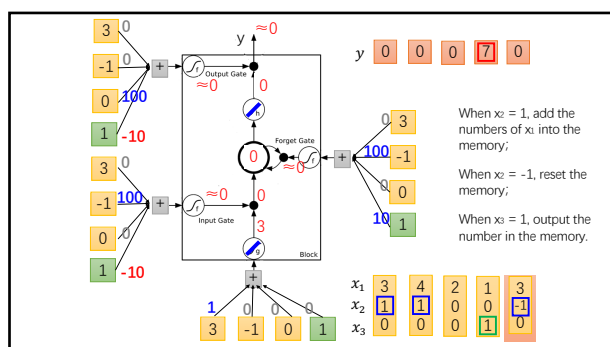
25



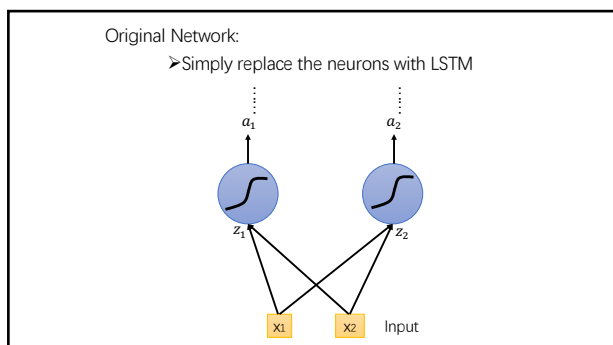
26



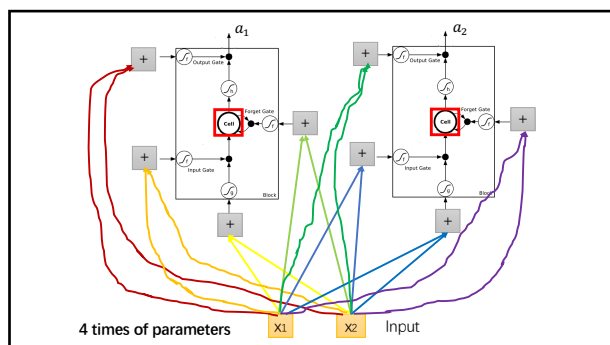
27



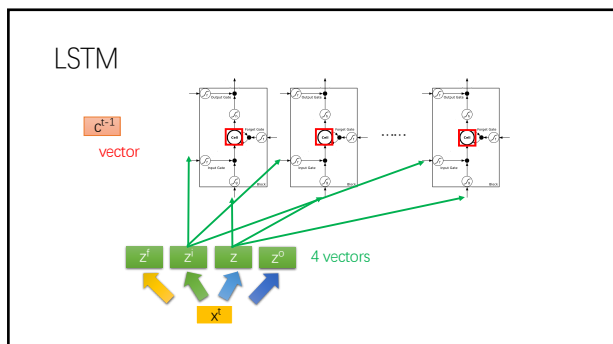
28



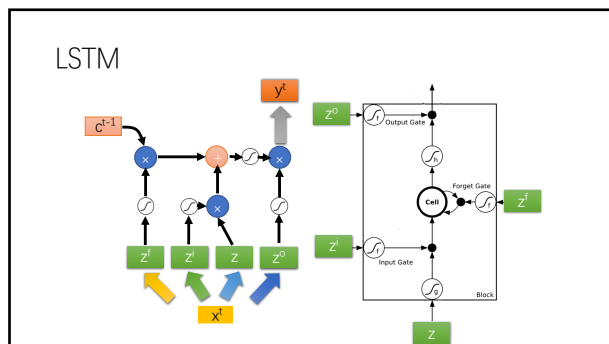
29



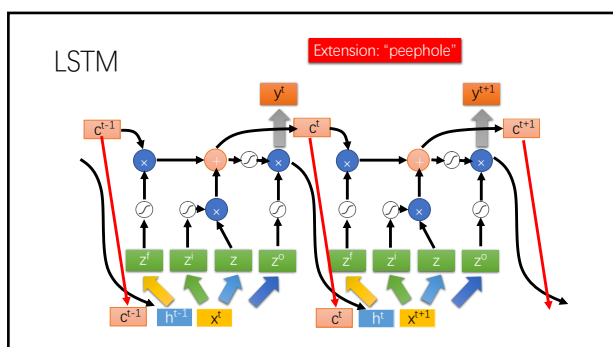
30



31



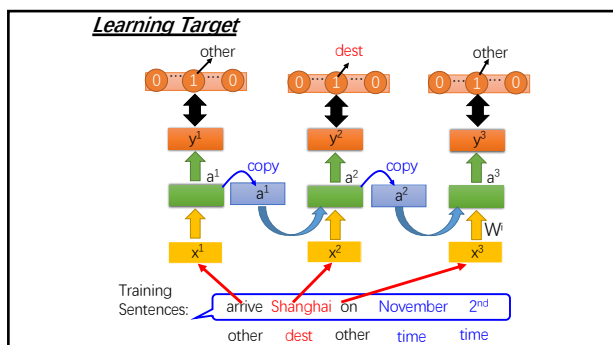
32



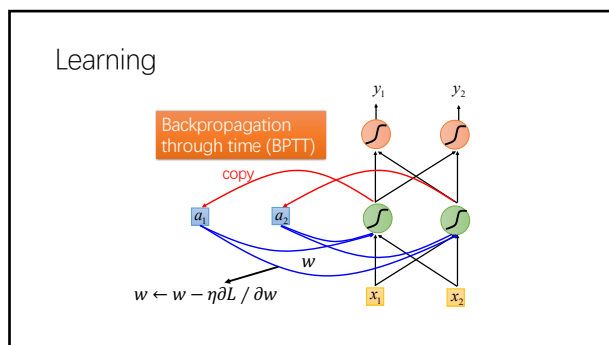
33



34



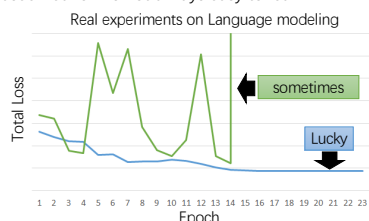
35



36

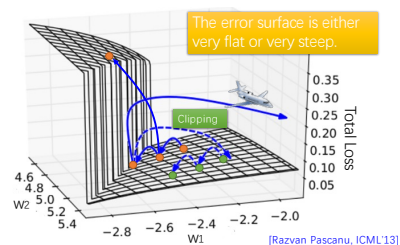
Unfortunately

- RNN-based network is not always easy to learn



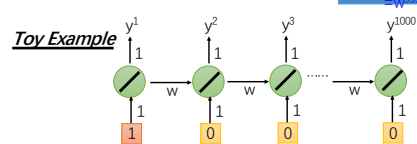
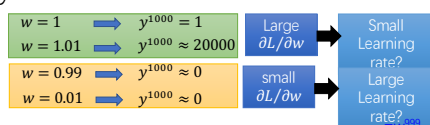
37

The error surface is rough.



38

Why?



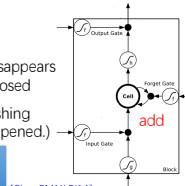
39

Helpful Techniques

Long Short-term Memory (LSTM)

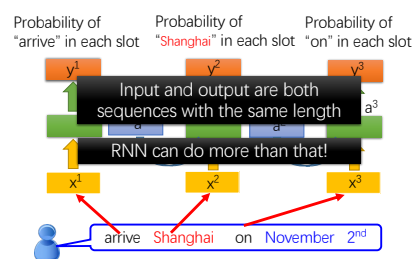
- Can deal with gradient vanishing (not gradient explode)
- Memory and input are **added**
- The influence never disappears unless forget gate is closed
- No Gradient vanishing (if forget gate is opened.)

Gated Recurrent Unit (GRU): simpler than LSTM



40

More Applications



41

准备NER数据

```

label_to_id:
{'TIM': 2, 'DES': 3, 'DEP': 4, 'O': 1, '<pad>': 0}
vocab (word_id):
{'arrive': 1, 'Shanghai': 2, 'on': 3, 'November': 4, '2nd': 5, 'and': 6, 'Leave': 7, '<pad>': 0}

# 数据
data = [
{"text": "arrive Shanghai on November 2nd and",
 "entities": [
  {"start": 7, "end": 15, "label": "DES"},
  {"start": 19, "end": 31, "label": "TIM"}]},
{"text": "leave Shanghai on November 2nd",
 "entities": [
  {"start": 6, "end": 14, "label": "DEP"},
  {"start": 18, "end": 30, "label": "TIM"}]}]

# 准备数据加载器
class NERDataset(Dataset):
    def __init__(self, data):
        self.data = data
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        return self.data[idx]

train_data:
[[[1, 2, 3, 4, 5, 6], [1, 3, 1, 2, 2, 1]],
 [[7, 2, 3, 4, 5], [1, 4, 1, 2, 2]]]

dataset = NERDataset(train_data)

```

42

准备NER数据

```
from torch.nn.utils.rnn import pad_sequence

def collate_fn(batch): #定义一个自定义的数据批次处理函数
    src_batch, trg_batch = zip(*batch) #将批次中的样本“转置”
    # 填充批次中的句子
    src_batch = pad_sequence([torch.tensor(s) for s in src_batch],
                             padding_value=0, batch_first=True)
    trg_batch = pad_sequence([torch.tensor(t) for t in trg_batch],
                             padding_value=0, batch_first=True)
    return src_batch, trg_batch

dataloader = DataLoader(dataset, batch_size=2, shuffle=True,
                        collate_fn=collate_fn)
```

43

定义NER模型

```
class NERModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers=1,
                             Bidirectional=False, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)
        lstm_out, _ = self.lstm(embedded)
        logits = self.fc(lstm_out)
        return logits
```

44

参数设置

```
# 初始化模型和优化器
VOCAB_SIZE = len(vocab)
EMBEDDING_DIM = 16
HIDDEN_DIM = 16
OUTPUT_DIM = len(label_to_id)

model = NERModel(VOCAB_SIZE, EMBEDDING_DIM, HIDDEN_DIM, OUTPUT_DIM)
criterion = nn.CrossEntropyLoss(ignore_index=0) #即不计padding
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

momentum 作用: 加速收敛并减少震荡
原理: 把之前的更新方向和当前梯度结合起来就像推一个下坡的球, 它会积累动量。
0.9 是常用值, 表示保留90%的历史方向。

45

语音识别概述

语音识别(Speech Recognition)技术, 也被称为自动语音识别(Automatic Speech Recognition, ASR)、电脑语音识别(Computer Speech Recognition)或是语音转文本识别(Speech To Text, STT), 其目标是以电脑自动将人类的语音内容转换为相应的文字。

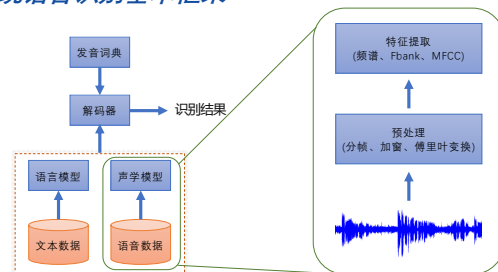
46

语音识别发展历史

- 1920年代生产的最早的语音识别器Radio Rex玩具狗
- 最早的基于电子计算机的语音识别系统由AT&T贝尔实验室开发的Audrey
- 1960年代, 人工神经网络被引入了语音识别
- 卡内基梅隆大学的李开复实现了第一个基于隐式马尔科夫(Hidden Markov Model, HMM)模型的大词汇量语音识别系统Sphinx
- 2012年, 微软研究院的俞栋和邓力等人提出了上下文相关的深度神经网络与隐马尔可夫模型融合的声学模型(CD-DNN-HMM)
- 近年来, 兴起了基于端到端(End-to-End, E2E)的语音识别方案

47

传统语音识别基本框架

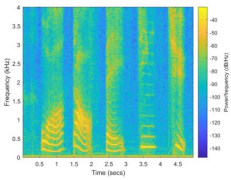


48

声音信号处理——分帧

自动语音识别
Automatic Speech Recognition

一段完整的语音信号通常是非稳态的，即其特征基本上随时间变化的，然而由于人的口腔与喉部发声的固有特性，再一个短时间段内，一般为20-50ms之间，其特征基本保持不变，即语音信号是短时平稳的。所以在对语音信号进行频域分析时需要一帧一帧地处理，帧长一般取20-50ms。



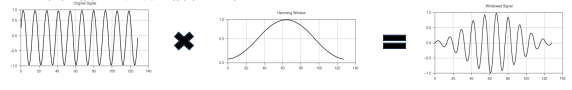
“我到北京去” 频谱图
频普特征随时间变化非常明显

49

声音信号处理——加窗

自动语音识别
Automatic Speech Recognition

加窗的目的是让每一帧信号的幅度在两端渐变到0，减轻频谱泄漏，便于傅里叶变换。取出来的每一帧信号，在做傅里叶变换之前，要进行加窗操作，即将每一帧信号与窗函数相乘。常见的窗函数有矩形窗、汉宁窗与汉明窗。下图为信号与汉明窗相乘的过程。



加窗的代价是每一帧信号两端被削弱，为弥补这一缺陷，分帧时帧与帧之间需要相互重叠一部分(overlap)。相邻两帧的起始位置的时间差叫做帧偏移，常见的取法为帧长的一半或者固定取为10ms。

50

声音信号处理——傅里叶变换

自动语音识别
Automatic Speech Recognition

傅里叶变换 (Fourier Transform) 是一种分析信号的方法，它可分析信号的成分，也可用这些成分合成信号。许多波形可作为信号的成分，傅里叶变换用正弦波作为信号的成分。

设 $f(t)$ 是关于时间 t 的函数，则傅里叶变换可以检测频率 ω 的周期在 $f(t)$ 出现的程度：

$$F(\omega) = \mathbb{F}[f(t)] = \int_{-\infty}^{\infty} f(t)e^{-j\omega t} dt$$

傅里叶变换相当于将时域的函数与周期为 2π 的复指数函数进行连续的积分。

傅里叶变换有相应的卷积定理，可以将时域的卷积转化为频域的乘积，也可以将频域的卷积转化为时域的乘积。

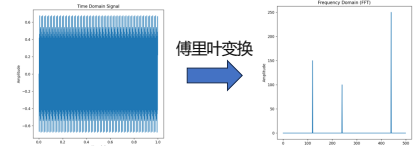
51

声音信号处理——傅里叶变换

自动语音识别
Automatic Speech Recognition

复合正弦波信号 (模拟声音)
频率1: 440 Hz (A4 音调)
频率2: 880 Hz (A5 音调)
频率3: 1760 Hz (A6 音调)
采样率: 1000Hz
信号时长: 1.0秒

傅里叶变换的主要目的是将时域信号转化为频域信号，便于分析、滤波、压缩和解决信号中的各种问题。在频域中，信号的频率成分一目了然，可以更容易地进行处理和信号分析。



傅里叶变换

52

声音信号处理——特征提取

自动语音识别
Automatic Speech Recognition

- 时域特征
时域特征直接从声音信号的时间波形中提取，是声音信号最基础的表示方法。这些特征在时间序列中描述信号的幅度变化。
- 频域特征
声音信号通过傅里叶变换可以从时域转换为频域表示，在频域中信号的频率成分清晰可见。频域特征在许多应用中非常重要，特别是对于音乐和语音分析。
- Fbank特征
Fbank特征通过将声音信号通过一组梅尔滤波器组来提取频谱能量，保留了信号中的频率分布信息，同时也模拟了人类听觉系统对频率的感知方式。
- MFCC
MFCC (Mel-frequency Cepstral Coefficients) 是语音处理中特别常用的一种特征提取方法。它利用人类听觉系统对频率的非线性感知特性，将声音信号转化为基于梅尔标度的倒谱特征。
- ...

53

语音识别——基本概率模型

自动语音识别
Automatic Speech Recognition

语音识别可以表示为给定一段语音信号 X ，求出最可能的对应词序列 W ，即：

$$W^* = \arg \max_W P(W|X)$$

这里的 W^* 是我们想要的最佳词序列， $P(W|X)$ 表示给定语音信号 X 的情况下，词序列 W 出现的条件概率。

根据贝叶斯定理，我们可以将条件概率 $P(W|X)$ 分解为：

$$P(W|X) = \frac{P(X|W)P(W)}{P(X)}$$

54

自动语音识别
Automatic Speech Recognition

语音识别——基本概率模型

$$P(W|X) = \frac{P(X|W)P(W)}{P(X)}$$

- $P(X|W)$: 声学模型(Acoustic Model), 表示在给定词序列 W 的情况下, 语音信号 X 的生成概率。
- $P(W)$: 语言模型(Language Model), 表示词序列 W 的先验概率。它反映了预言的语法和结构信息。
- $P(X)$: 归一化项, 表示语音信号 X 本身的概率。在识别过程中, $P(X)$ 是一个常数, 因此我们可以将其忽略, 只需最大化 $P(X|W)P(W)$ 。

最终, 我们的目标变为在所有可能的词序列中找到一个使声学模型和语言模型乘积最大的序列, 即:

$$W^* = \arg \max_W P(X|W)P(W)$$

55

自动语音识别
Automatic Speech Recognition

语音识别——声学模型

声学模型(Acoustic Model), 描述了语音信号和词汇之间的关系。它主要负责将连续的语音信号(通常以特征如MFCC或Fbank表示)映射到离散的词或音素序列。

典型的声学模型包括基于隐马尔可夫模型(Hidden Markov Model, HMM)和高斯混合模型(Gaussian Mixture Model, GMM)的建模方法。HMM是一种经典的概率模型, 特别适用于处理时序数据。HMM 假设语音信号可以用一系列的隐状态表示, 音素或词的发音是这些隐状态的输出。

GMM是一种统计模型, 用于表示具有多种潜在分布的复杂数据集。它假设数据点是由多个高斯分布的线性组合生成的, 适合于处理具有多个聚类或模式的情况。

基于HMM-GMM的混合模型在很长一段时间内是语音识别的主流。

56

RNN和语音识别
RNN and Automatic Speech Recognition

语音识别——声学模型

随着深度学习技术的发展, 声学模型从传统的GMM逐渐过渡到基于深度神经网络(Depth Neural Network, DNN)的模型。以下是语音识别中常见的几类深度学习声学模型:

- 深度神经网络(Depth Neural Network, DNN)
- 卷积神经网络(Convolutional Neural Network, CNN)
- 循环神经网络(Recurrent Neural Network, RNN)
- 长短期记忆网络(Long Short-Term Memory, LSTM)
- 时间卷积网络(Temporal Convolutional Network, TCN)
- Transformer模型(Attention机制)
- Conformer模型(CNN+Transformer)

57

RNN和语音识别
RNN and Automatic Speech Recognition

输入 Acoustic Feature

length T, dimension d

10ms 1s → 100 frames

25ms

frame

400 sample points (16KHz)
39-dim MFCC
80-dim filter bank output

數位語音處理 第七章
Speech Signal and Front-end Processing
<http://ocw.aca.ntu.edu.tw/ntu-ocw/ocw/cou/1045204/7>

58

RNN和语音识别
RNN and Automatic Speech Recognition

输入

Acoustic Feature

Waveform

DFT

spectrogram

DCT

log

MFCC

filter bank

59

RNN和语音识别
RNN and Automatic Speech Recognition

输出

Token

Phoneme: a unit of sound

W AH N P AH N CH MA EN
one punch man

Lexicon: word to phonemes
cat → K AE T
good → G UH D
man → MA EN
one → W AH N
punch → P AH N CH

Grapheme: smallest unit of a writing system

one_punch_man
N=13, V=26+?

26 English alphabet
+ { } (space)
+ [punctuation marks]

Chinese does not need "space"

60

CTC

RNN和语音识别
RNN and Automatic Speech Recognition

连接时序分类模型(Connectionist Temporal Classification, CTC)

设音频信号长度为T, 目标文本序列长度为U, 通常情况下T>U。
传统的序列到序列模型, 如HMM, 需要输入序列与输出序列对齐, 即T=U。
CTC能够自动推断输入与输出之间的关系, 不需要手动标注主对齐。
CTC在原有的词汇表基础上, 增添了一个特殊空白符, 假设为<blank>

67

语音识别——端到端模型End2End

RNN和语音识别
RNN and Automatic Speech Recognition

假设我们从音频中提取了5个时间步的特征向量

$$X = (x_1, x_2, x_3, x_4, x_5)$$

假设我们的词汇表大小为4(包含<blank>)模型对每个时间步进行预测后, 得到了一个5*4大小的概率矩阵, B代表<blank>, 目标序列为CAT

时间步	C	A	T	B
x_1	0.1	0.6	0.1	0.2
x_2	0.7	0.1	0.1	0.1
x_3	0.1	0.1	0.7	0.1
x_4	0.1	0.2	0.6	0.1
x_5	0.6	0.1	0.1	0.2

68

语音识别——端到端模型End2End

RNN和语音识别
RNN and Automatic Speech Recognition

CTC会合并相同的字符, 忽略<blank>, 因此我们有多种路径可以最终合并为CAT, 例如:
BCATB, BCATT, CCATB, CCAAT

对于BCATB, 其概率为 $P(BCATB) = P(B|x_1) \cdot P(C|x_2) \cdot P(A|x_3) \cdot P(T|x_4) \cdot P(B|x_5)$
代入表中数据可得 $P=0.2 \cdot 0.7 \cdot 0.1 \cdot 0.6 \cdot 0.2 = 0.00168$

时间步	C	A	T	B
x_1	0.1	0.6	0.1	0.2
x_2	0.7	0.1	0.1	0.1
x_3	0.1	0.1	0.7	0.1
x_4	0.1	0.2	0.6	0.1
x_5	0.6	0.1	0.1	0.2

69

语音识别——端到端模型End2End

RNN和语音识别
RNN and Automatic Speech Recognition

CTC通过前向-后向算法动态规划来高效地计算所有可能路径的总和。
CTC的损失是所有可能路径的概率总和的负对数。假设计算出的所有路径概率总和为 $P(CAT|X) = 0.00252$ 。
那么CTC损失为 $CTC\ Loss = -\log(P(CAT|X)) = -\log(0.00252) \approx 5.99$
通过计算出CTC损失, 模型使用反向传播更新网络的参数, 以提高生成目标序列 "CAT" 的概率。训练的目标是让模型在每个时间步上预测的字符概率更加接近实际的目标序列。

70

语音识别——评价指标

WER(Word Error Rate)

WER 是衡量语音识别系统性能的标准指标, 表示识别的单词错误率。WER 计算的是识别结果与目标参考文本在词级别的差异, 具体通过编辑距离(Levenshtein距离)来衡量两个文本之间的不同操作数量(插入、删除和替换)。计算公式如下:

$$WER = \frac{S + D + I}{N}$$

S: 替换的词数 —— 将一个词错误地识别为另一个词
D: 删除的词数 —— 参考文本中的词被识别系统遗漏
I: 插入的词数 —— 识别系统多识别出的词
N: 参考文本中的总词数

71

语音识别——评价指标

WER(Word Error Rate)

计算示例:
识别结果 Hypothesis: "the fast brown dog"
参考文本 Reference: "the quick brown fox"
替换: quick——fast fox——dog
无删除和插入

$$WER = \frac{2 + 0 + 0}{4} = 0.5$$

72

语音识别——评价指标

CER(Character Error Rate)

CER 类似于 WER, CER 在字符级别计算语音识别错误率,而不是在词级别。CER 也是通过编辑距离来计算参考文本和识别输出之间的差异。计算公式如下:

$$CER = \frac{S + D + I}{N}$$

73

语音识别——评价指标

CER(Character Error Rate)

计算示例:

识别结果 Hypothesis: "bat"

参考文本 Reference: "cat"

替换: c→b

无删除和插入

$$CER = \frac{1 + 0 + 0}{3} = 0.33$$

74

语音识别——评价指标

WER 与 CER 的对比

WER 关注的是词级别的错误率,而 CER 关注的是字符级别的错误率。通常 CER 会提供更精细的错误分析,尤其是在字符级别的语言(如中文、日文等)中。

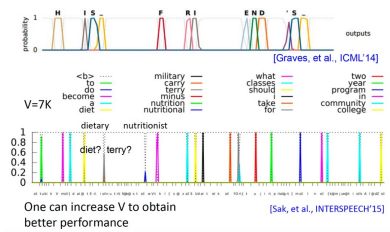
对于西方语言,如英语,WER 更常用,因为词是主要的识别单位。而在像中文这样的语言中,CER 更有用,因为中文没有明确的词分界,字符级别的评估更具代表性。

在某些情况下,WER 会高于 CER,因为一个词级别的错误可能包含多个字符级别的错误。此外,CER 通常用于计算较短序列的错误率,而 WER 对长序列和整句错误有更好的分析。

如果对文本进行更细粒度的划分,如利用发音词典将字符转化为音素,则可以计算PER,即 Phones Error Rate。

75

Does CTC work?



76

Does CTC work?

Model	CER	WER
Encoder-Decoder	6.4	18.6
Encoder-Decoder + bigram LM	5.3	11.7
Encoder-Decoder + trigram LM	4.8	10.8
Encoder-Decoder + extended trigram LM	3.9	9.3
Graves and Jaitly (2014)		
CTC	9.2	30.1
CTC, expected transcription loss	8.4	27.3
Hamn et al. (2014)		
CTC	10.0	35.8
CTC + bigram LM	5.7	14.1
Miao et al. (2015)		
CTC for phonemes + lexicon	-	26.9
CTC for phonemes + trigram LM	-	7.3
[Bahdanau, et al., ICASSP'16]		
CTC + trigram LM	-	9.0

80 hours

77

Does CTC work?

Method	#Params (M)	WER Without LM		WER With LM	
		testclean	testother	testclean	testother
Hybrid					
Transformer [33]	-	-	-	2.26	4.85
CTC					
QuartzNet [9]	19	3.90	11.28	2.69	7.25
LAS					
Transformer [34]	270	2.89	6.98	2.33	5.17
Transformer [19]	-	2.2	5.6	2.6	5.7
LSTM	360	2.6	6.0	2.2	5.2
Transducer					
Transformer [7]	139	2.4	5.6	2.0	4.6
ContextNet(S) [10]	10.8	2.9	7.0	2.3	5.5
ContextNet(M) [10]	31.4	2.4	5.4	2.0	4.5
ContextNet(L) [10]	112.7	2.1	4.6	1.9	4.1
Conformer (Ours)					
Conformer(S)	10.3	2.7	6.3	2.1	5.0
Conformer(M)	30.7	2.3	5.0	2.0	4.3
Conformer(L)	118.8	2.1	4.3	1.9	3.9

970 hours

Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. arXiv preprint arXiv:2005.08100, 2020.

78

定义CTC模型

```
class CTC_ASR_Model(torch.nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, dropout=0.1):
        super(CTC_ASR_Model, self).__init__()
        self.lstm = torch.nn.LSTM(input_dim, hidden_dim, num_layers=2,
                                   batch_first=True, bidirectional=True, dropout=dropout)
        self.dropout = torch.nn.Dropout(dropout)
        self.fc = torch.nn.Linear(hidden_dim * 2, output_dim)

    def forward(self, x):
        x, _ = self.lstm(x)
        x = self.dropout(x)
        x = self.fc(x)
        return x
```

79

定义CTC模型-2

```
# 初始化模型
input_dim = 80 # 梅尔频谱图的特征维度
hidden_dim = 256 # 隐藏层维度
output_dim = len(vocab)*128 # 输出维度 (字符集大小)
model = CTC_ASR_Model(input_dim, hidden_dim, output_dim).to(device)

# 定义损失函数和优化器
criterion = nn.CTCLoss(blank=0, reduction='mean', zero_infinity=True)
optimizer = optim.Adam(model.parameters(), lr=0.001)#, weight_decay=0.01
```

80